Centro Universitário de Excelência
Sistemas de Informação

MecâniQA Tech — OAT 2

Autores: Paulo Henrique

Kauã Henrique
João Henrique

2026

Agenda

Comprovar como a MecâniQA monitorou, escalou e documentou o ambiente IoT.

1. Arquitetura

IoT + Kubernetes

2. Monitoramento
Prometheus + Grafana

3. Escalabilidade
HPA e teste real

4. Alertas
Quatro regras

5. Continuidade
Runbooks e metas

Arquitetura observável

- IoT Python + Kubernetes + Prometheus
- Grafana: painéis e alertas operacionais

Simulador IoT

- Instrumentação em Python:
  - Contadores de erros/sucessos
  - Métricas na porta 8000
  - POST de leituras em 8001

PYTHON | Instrumentação

```
sucessos = Counter(
    "mecaniqa_leituras_sucesso_total",
erros = Counter(
    "mecaniqa_leituras_erro_total",
start_http_server(8000)
servidor_iot = ThreadingHTTPServer(
    ("0.0.0.0", 8001),
    LeituraHandler
)
```

FONTE: iot/simulador.py

Coleta Prometheus

- Monitoramento do cluster:
  - Scrape a cada 5 segundos
  - Descoberta de Pods por DNS
  - Grafana conectado à fonte

PROMETHEUS | Scrape

```
global:
  scrape_interval: 5s
scrape_configs:
  - job_name: "mecaniqa-iot"
    dns_sd_configs:
      - names:
          - mecaniqa-iot-metrics...
        port: 8000
```

FONTE: k8s/prometheus-configmap.yaml

Painéis do Grafana

DASHBOARD OAT 2 — CONFIGURAÇÃO EXPORTADA

01 CPU | Python

Consulta PromQL:

```
100 * rate(process_cpu_seconds
_total{job="mecaniqa-iot"}[2m])
```

02 MEMORIA | Python

Consulta PromQL:

```
process_resident_memory
_bytes{job="mecaniqa-iot"} / 1024
 / 1024
```

03 ERROS | IoT

Consulta PromQL:

```
rate(mecaniqa_leituras_erro
_total[2m])
```

04 SUCESSOS | IoT

Consulta PromQL:

```
rate(mecaniqa_leituras_sucesso
_total[2m])
```

FONTE: grafana/dashboard-oat2.json | Os gráficos numéricos não foram recriados.

HPA configurado

- Escalonamento automático:
  - Meta: CPU 60% e memória 70%
  - Mínimo 1 e máximo 4 Pods
  - Métricas: Metrics Server

KUBERNETES | autoscaling/v2

```
minReplicas: 1
maxReplicas: 4
metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        averageUtilization: 60
```

FONTE: k8s/iot-hpa.yaml

Teste de estresse

- Evidência em 21/10:
  - 16 trabalhadores / 240 s
  - 39.406 HTTP sem exceção
  - Escalonamento: 1 → 4 Pods

EVIDÊNCIA | HPA

```
> cpu: 587%/60%, memory: 44%/70%
> Deployment pods: 4 current / 4 desired
> SuccessfulRescale
> New size: 4; reason: cpu above target
> TESTE FINALIZADO
> sucessos HTTP: 39406
> falhas_http: 14
```

FONTE: evidencias/oat2/21-10/03-teste-estresse-hpa.txt

Alertas do Grafana

- Quatro regras configuradas:
  - CPU, memória e erros: Firing*
  - Sucessos: regra ativa
  - * Observado em aula, sem log

GRAFANA | Limites configurados

```
CPU         > 70% do request / 1 min
Memoria     > 80% do request / 1 min
Erro IoT    > 10 leituras/s / 1 min
Sucessos    < 0,1 leitura/s / 2 min

4 regras ativas no JSON versionado
3 estados Firing relatados na aula
```

FONTE: grafana/criar_alertas_oat2.py

Continuidade e Runbooks

- Procedimentos documentados:
  - 4 Runbooks de diagnóstico
  - SLA exigido: 99,9%
  - RTO/RPO: propostas a validar

RUNBOOKS | Diagnostico

```
$ kubectl get pods -l app=mecaniqa-iot
$ kubectl top pods -l app=mecaniqa-iot
$ kubectl describe hpa mecaniqa-iot-hpa
$ kubectl logs deploy/mecaniqa-iot

Acao e verificacao: conforme incidente
Procedimentos propostos, nao testados
```

FONTE: relatorio tecnico consolidado

Considerações finais

- Escalabilidade comprovada: 1 → 4 Pods no HPA.
- Quatro alertas criados; três observados em Firing.
- Próximo passo: validar metas e exercícios de recuperação.

Referências

- GitHub: PauloHenri11/mecaniQA-belo-horizonte (main).
- OAT 2 — Caderno de Guias: Monitorando a Continuidade.
- Relatório Técnico Consolidado — MecâniQA OAT 2, 2026.